Unreal Engine 5 Chaos Simulation Specification

Technical Guide to Native C++ ActorComponents, Ground-Truth Raycasting, and Control Arbitration

METRIC / SCOPE	SPECIFICATION / TARGET
Subsystem: Unreal Engine 5 Simulation	Module Path: ue_sim/ue5_adapter & SAFAR_CPP_Sim
Engine Target: Unreal Engine 5.5	Physics Model: Chaos Wheeled Vehicles
Control Policy: Throttle Clamp + Brake Blend	Steering Lock: 0% (Fully Preserved)

1. Subsystem Architecture & Chaos Vehicle Integration

The Unreal Engine 5 Simulation Subsystem integrates SAFAR directly into Epic Games' Chaos Wheeled Vehicle framework. It provides native C++ ActorComponents attached to vehicle pawns, ground-truth obstacle perception, ground line-tracing pothole scanners, and a player controller that resolves driver vs. ADAS control fighting.

2. Player vs. Safety Control Arbitration Fix

A critical engineering achievement of SAFAR in Unreal Engine is the complete resolution of the control fight:
• **Throttle Clamping:** When `IsAutonomousOverrideActive()` is true, the driver's throttle key (W) is clamped to `0.0f`.
• **Safe Brake Blending:** Uses `FMath::Max(PlayerBrake, SafarBrake)` so the driver can always brake harder than SAFAR.
• **Uncontested Steering:** Driver retains 100% manual steering authority at all times (SAFAR never locks the wheel).

3. Source Code Files Breakdown

FILE NAME	RESPONSIBILITY & ARCHITECTURAL WORK
ue5_adapter/Source/SAFARVehicleComponent.h/.cpp	Master C++ vehicle coordinator. Ticks perception, feeds TargetTracker, runs ThreatEngine, and applies brake/throttle to Chaos Movement.
ue5_adapter/Source/GroundTruthPerception.h/.cpp	Raycast perception provider. Fires multi-point line traces and pawn sphere sweeps up to 50m to gather ground truth obstacles.
ue5_adapter/Source/UEPotholeScanner.h	Ground line-tracing component. Scans road surface ahead up to 30m to detect depth void depressions and potholes.
ue5_adapter/Source/PlayerVehicleController.h/.cpp	Player controller implementing control arbitration: silences throttle during emergency intervention while preserving steering.
ue5_adapter/Source/SAFARHUDWidget.h/.cpp	In-game driver warning overlay. Renders threat level (SAFE, ASSESSING, CRITICAL), target distance, and TTC.
SAFAR_CPP_Sim project (Source/SAFAR/)	Standalone Unreal Engine 5.5 project containing `USAFARSafetyComponent`, `ASAFARDemoHazard`, and `ASAFAR_CPP_SimPawn`.
live_runner/live_runner.py	High-speed Python telemetry loop connecting external perception or logging tools to UE5 over UDP sockets.

4. Line-by-Line Code Walkthrough: Chaos Control Arbitration

```
void APlayerVehicleController::VehicleThrottle(const FInputActionValue& Value)
{
    float ThrottleInput = Value.Get<float>();
    // If SAFAR autonomous override is active, clamp driver throttle to ZERO
    if (CachedSAFARComponent && CachedSAFARComponent->IsAutonomousOverrideActive())
    {
        ThrottleInput = 0.0f;
    }
    ChaosVehicleMovement->SetThrottleInput(ThrottleInput);
}

void APlayerVehicleController::VehicleBrakeTriggered(const FInputActionValue& Value)
{
    float PlayerBrake = Value.Get<float>();
    float SafarBrake = CachedSAFARComponent ? CachedSAFARComponent->GetSafarBrakeInput() : 0.0f;
    // Player can always brake harder than SAFAR, never softer
    float EffectiveBrake = FMath::Max(PlayerBrake, SafarBrake);
    ChaosVehicleMovement->SetBrakeInput(EffectiveBrake);
}

void APlayerVehicleController::VehicleSteering(const FInputActionValue& Value)
{
    // 100% Player steering authority is preserved at all times (SAFAR does not steer)
    ChaosVehicleMovement->SetSteeringInput(Value.Get<float>());
}
```

Code Explanation: Lines 5-9 silence the driver's throttle input during active emergency braking. Lines 15-18 blend brake inputs so the human driver can apply maximum braking effort without fighting the safety system. Lines 22-25 pass player steering directly to Chaos, allowing evasive swerves at any moment.